

Foundations of R Chapter 3

Robert Gulotty

9/30/2018

Graphical Summary of Data

Chapter 1 introduces R as a data manipulation engine. Chapter 2 as statistical analysis software. This chapter covers visualization in R using the popular ggplot2 package.

```
library(ggplot2)
library(dplyr, warn.conflicts = FALSE)
afg <- read.csv("~/Dropbox/mathCamp/Code/data/afg2.csv", as.is = T)
```

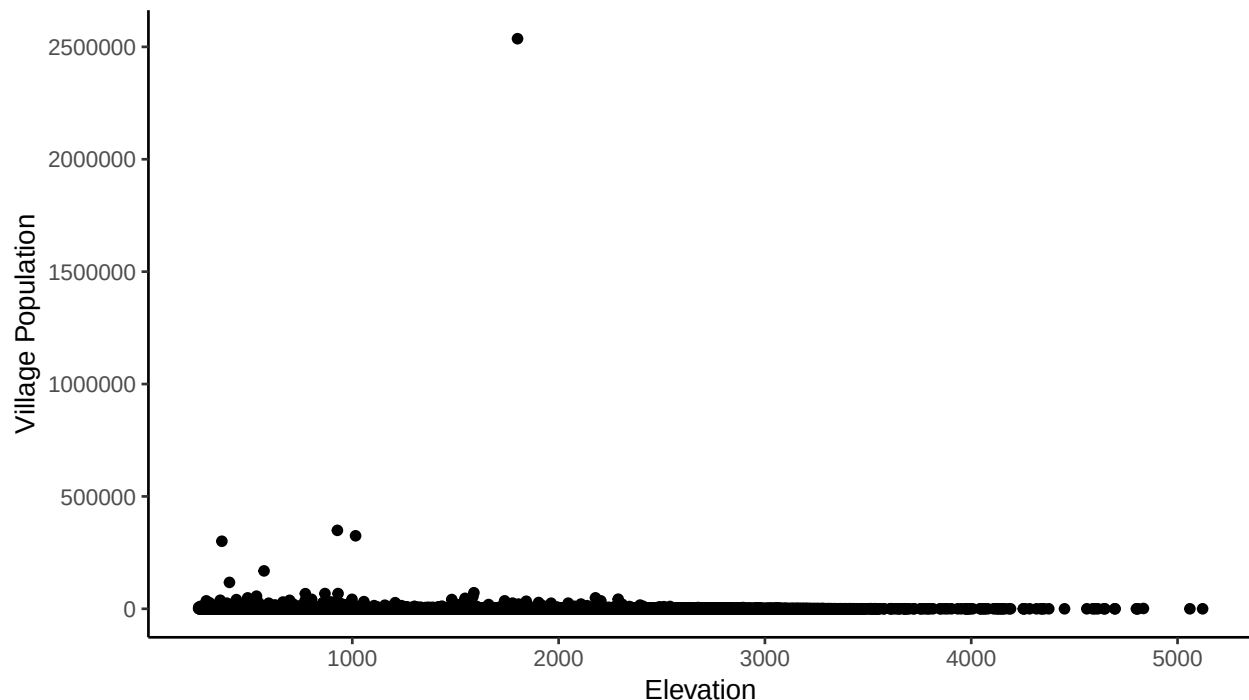
We will learn about plotting using a comprehensive dataset of villages in Afghanistan. This dataset has 45017 villages in Afghanistan. It records the Village Name, as well as other characteristics:

- CENTER: whether it is the District, or Provincial Capital
- POPULATION: The recorded population of that village
- LANGUAGE: What the majority language of that village is,
- ELEVATION: The elevation in meters.
- LAT_Y: The latitude
- LON_X: The longitude

Lets use graphical methods to understand the POPULATION variable.

The most basic way to visualize a variable is to plot the points. Here we use ggplot, which first takes on a data frame, then an aesthetic (aes) to put on the x and y axis. Then we add points to the plot with the geom_point command. Finally we add labels and transform the theme. Each of these are **layers**. ggplot places layers with +.

```
ggplot(afg, aes(ELEVATION, POPULATION)) +
  geom_point() +
  labs(x="Elevation", y="Village Population")+
  theme_classic()
```



We immediately notice that there is one large village, as well as many many small ones.

GGplot contains many customizable options that can improve data displays. One of the most common problems is overplotting, where many plots fall on top of one another. Here we introduce two common tricks. Taking the log and setting a transparency.

```
ggplot(afg, aes(ELEVATION, log(POPULATION + 1))) +  
  geom_point(alpha = .2)
```

Why do we use the log? What happens when population is 0? R doesn't try to plot this number. In general, it is helpful to use the log when the variable ranges more than a factor of ten, say if the data ranges from 10 to 1000.

Notice that the alpha is not inside of the `aes()` part of the ggplot command. Whenever you have a specific value for plot feature, it goes outside the aes. You can also make the alpha be a function of the data, which requires putting it in the aes command.

```
ggplot(afg, aes(ELEVATION, log(POPULATION + 1))) +  
  geom_point(aes(alpha = ELEVATION/max(ELEVATION)))
```

Changing the size can also help, which can be done either for every point or in a way that depends on another variable. This allows you to plot in $\mathbb{R}^3$, where the third dimension is size. Note again, size is inside the `aes`.

```
ggplot(afg, aes(ELEVATION, POPULATION, size = log(POPULATION + 1))) + geom_point(alpha = .2)
```

How would we set size to a particular number?

We can also change the x and y limits to focus the plot on a subset of the data.

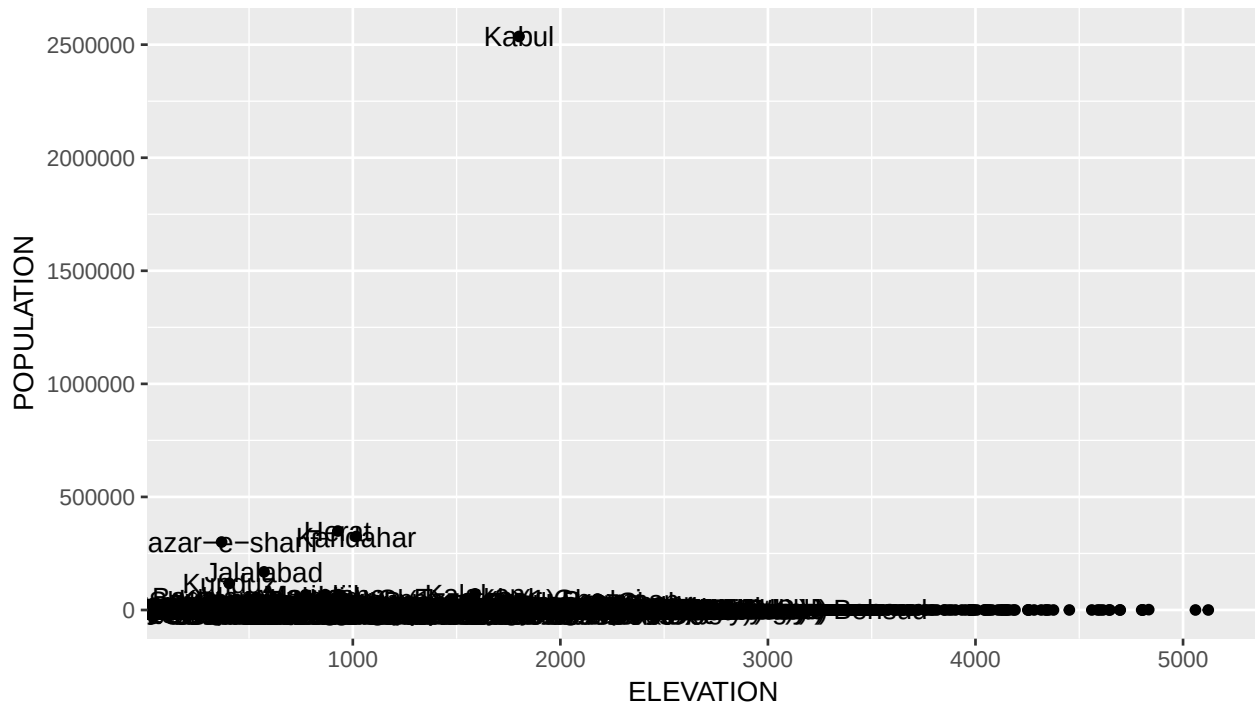
```
ggplot(afg, aes(ELEVATION, POPULATION)) +  
  geom_point(alpha = .2) + ylim(c(0, 3000))
```

There is one large village. We can use a filter command from dplyr to identify the row. Try that now:

```
filter(afg, POPULATION > 10000 )
```

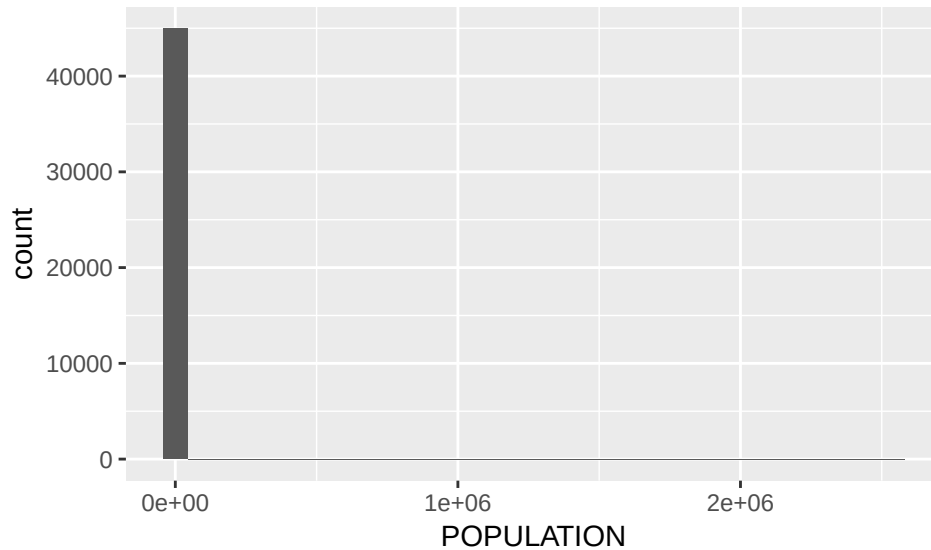
Again, the ggplot command uses a dataframe in its first argument, here `afg`, and then adds layers or `geom_` commands. The `aes` stands for the aesthetic, and sets the columns you want projected onto the x and y axis, or onto other aspects of the plot. Each layer then uses these same columns, or you can specify other columns within the geom layers. However, we can also change the data that the layers use to only apply them to a subset of the data, or to other data entirely. Here we add a `geom_text` to add a label to the points, but instead of labeling all 45 thousand villages, lets just label those that have a population greather than a threshold of 2000.

```
ggplot(afg, aes(ELEVATION, POPULATION, label = VILLAGE_NAME_ENG)) + geom_point() +  
  geom_text(data = filter(afg, POPULATION > 2000))
```



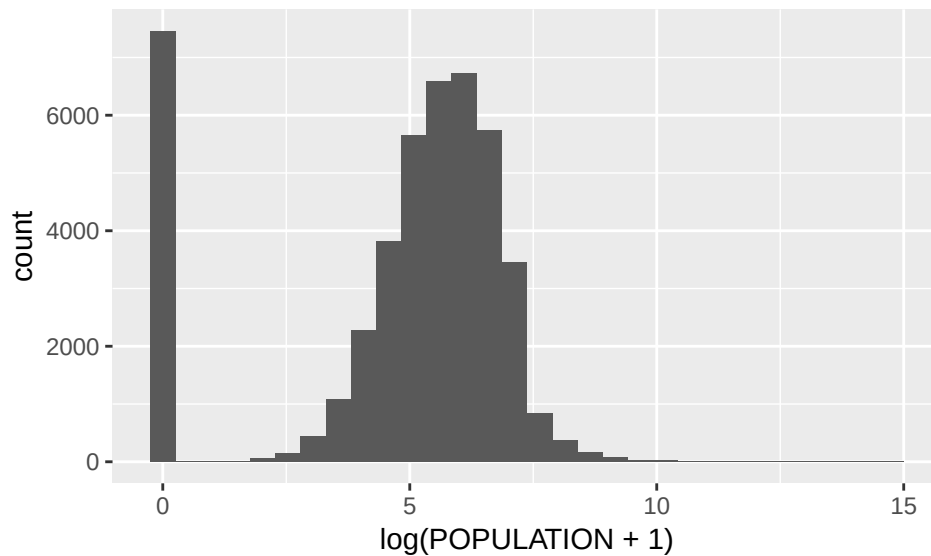
Another measurement strategy is to use the histogram geom.

```
ggplot(afg, aes(POPULATION)) + geom_histogram(bins=30)
```



Again, this doesn't help much, but a trick is to use the log:

```
ggplot(afg, aes(log(POPULATION + 1))) + geom_histogram(bins = 30)
```

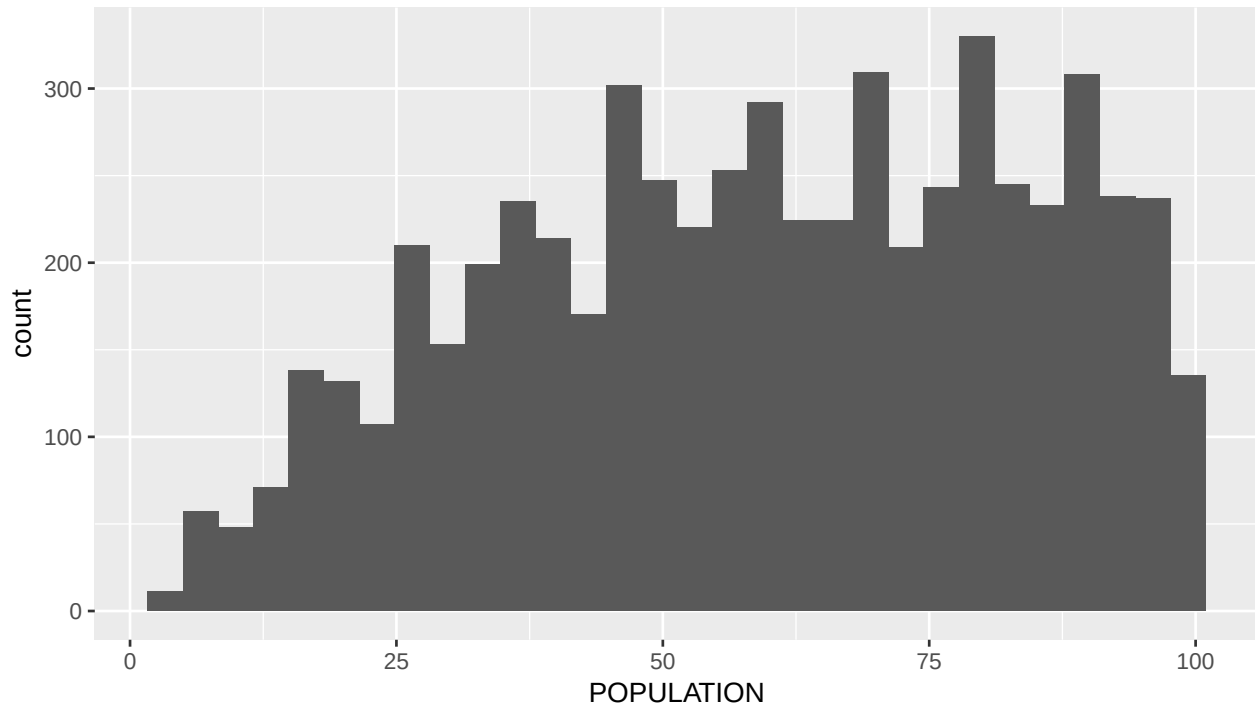


It seems like there is something going on here. We have a big bar at 0, and then a relatively normal distribution. This could be because villages cannot survive below a certain population.

One way to examine data is to subset to the relevant population. Here we use a filter command to see how big the smallest non-zero districts are. Here we combine a dplyr command (`filter`) with the ggplot commands.

```
afg%>%  
  filter(POPULATION > 0 & POPULATION < 100) %>%  
  ggplot(aes(POPULATION)) + geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

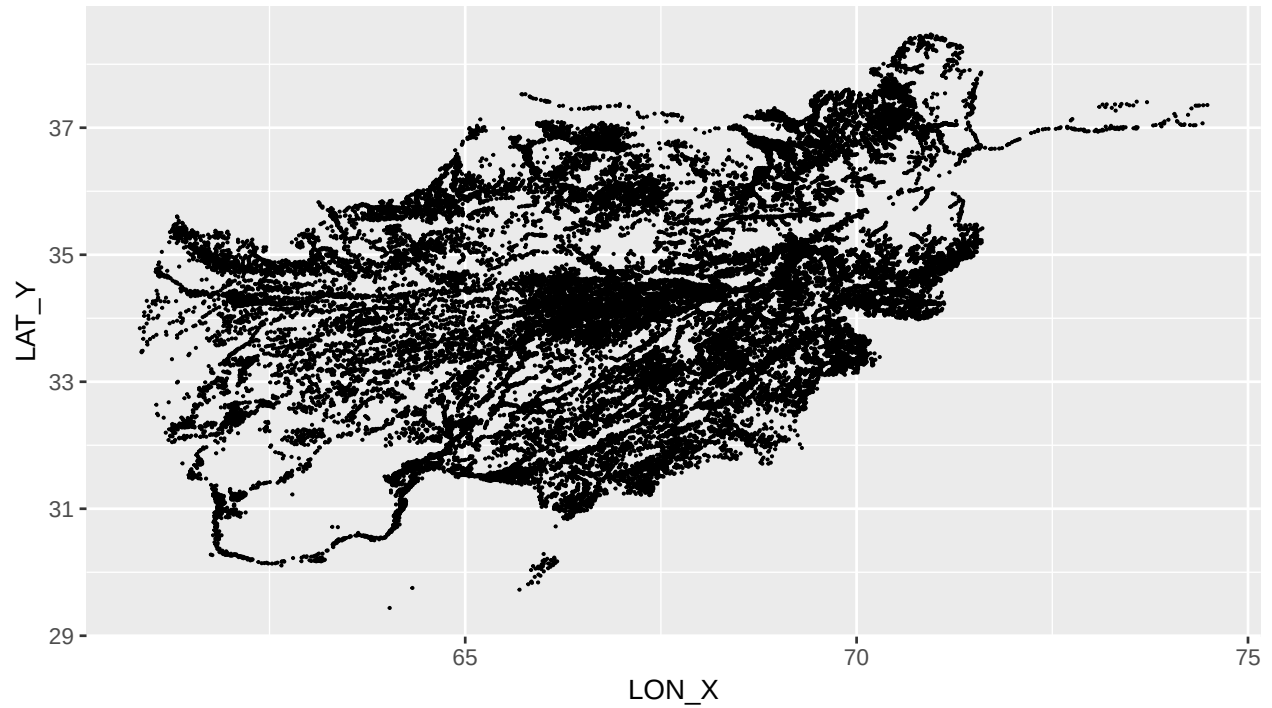


Seems like there are dozens of villages with less than 10 people in them.

Latitude and Longitude

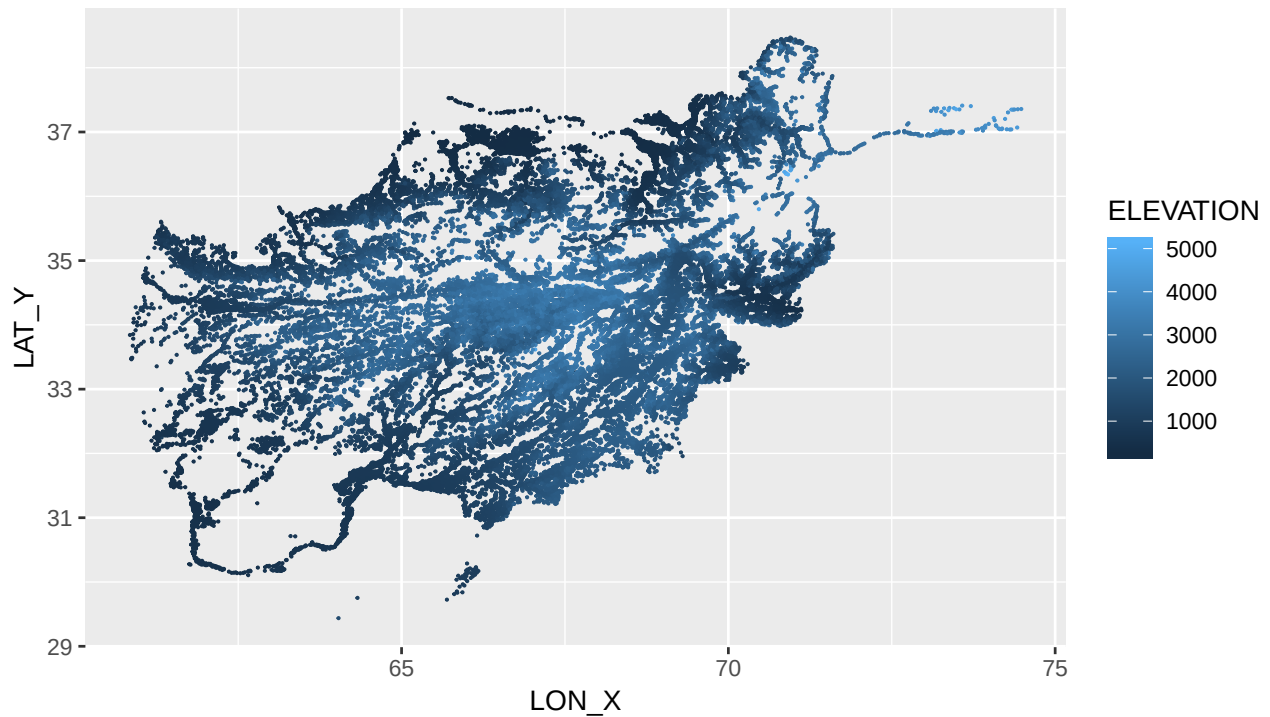
OK, so now that we know a little bit about population, lets take a look at the latitude and longitude variables.

```
ggplot(afg, aes(LON_X, LAT_Y)) + geom_point(size = .1)
```



We can adding a color to the plot by ELEVATION. Here we are specifying the color aesthetic. This can be in the top part of the function, or it can be placed in the individual layers. See the following are identical.

```
ggplot(afg, aes( LON_X,LAT_Y, color=ELEVATION))+geom_point( size=.1)
```

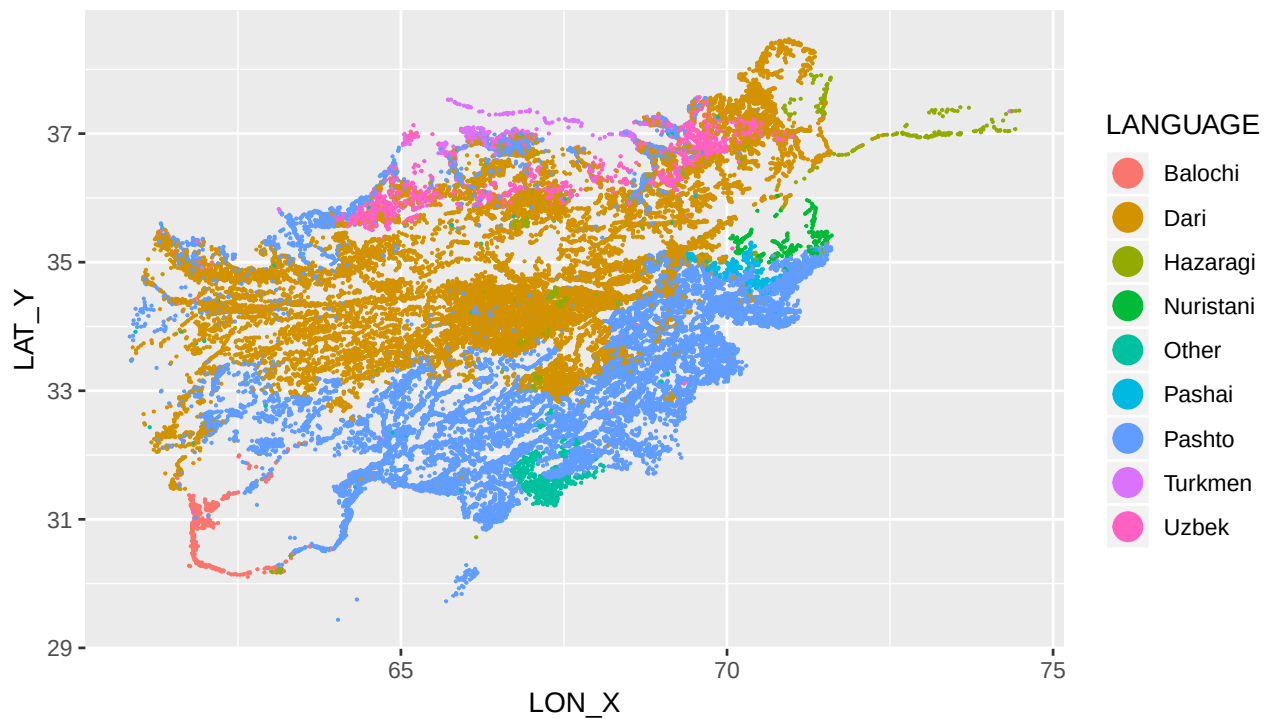


```
#ggplot(afg)+geom_point(aes(LON_X,LAT_Y, color=ELEVATION), size=.1)
#ggplot(afg, aes( LON_X, color=ELEVATION))+geom_point(aes(y = LAT_Y, color = ELEVATION), size=.1)
#ggplot(afg, aes( LON_X, LAT_Y, color=ELEVATION))+geom_point(aes(x = LAT_X, color = ELEVATION), size=.1)
```

Test your coloring skills: Try adding a color to the plot by $\log(\text{POPULATION}+1)$ and LANGUAGE.

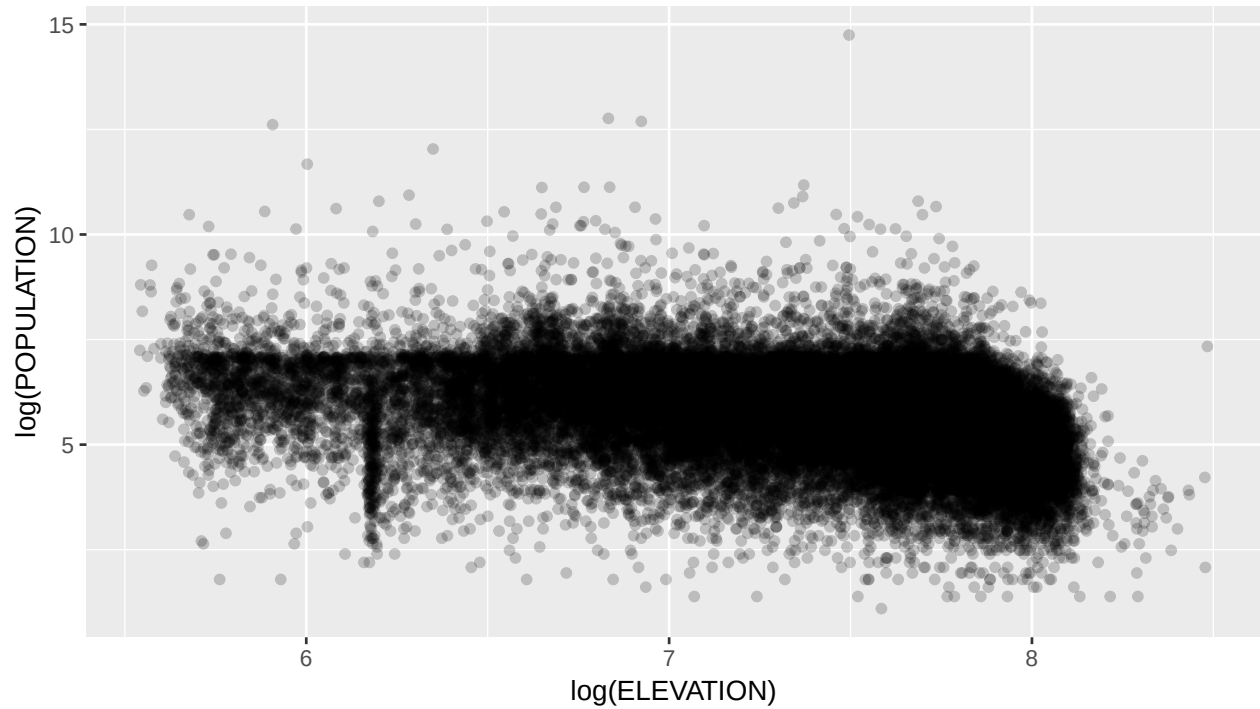
Note tha the legend in the guide, particularly for LANGUAGE is hard to read. We need to override the legend to make it bigger.

```
ggplot(afg, aes(LON_X, LAT_Y, color = LANGUAGE)) +  
  geom_point(size = .1) +  
  guides(color = guide_legend(override.aes = list(size = 5)))
```



Lets take a look at population and elevation again:

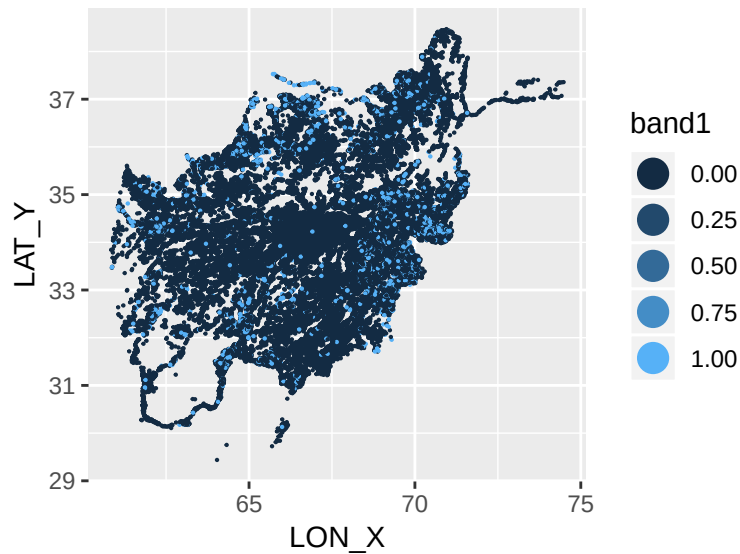
```
filter(afg, POPULATION>0)%>%  
ggplot( aes(log(ELEVATION), log(POPULATION)))+  
  geom_point(alpha=.2)
```



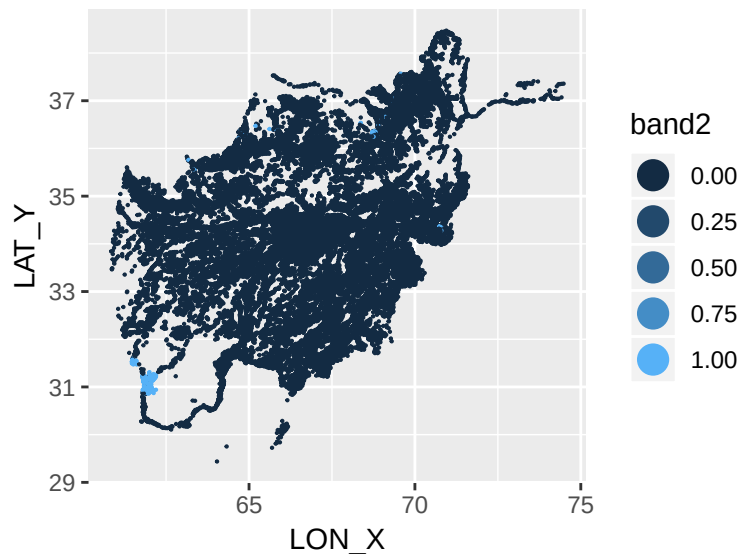
What is up with these bands? There seems to be a horizontal band at a little over 7 and a vertical band at 6.2 or so.

```
afg <- afg %>%
  mutate(band1 = ifelse(log(POPULATION) > 7 &
    log(POPULATION < 7.15), 1, 0),
    band2 = ifelse(log(ELEVATION) > 6.17 &
    log(ELEVATION) < 6.2, 1, 0))

ggplot(afg, aes(LON_X, LAT_Y, color = band1)) + geom_point(size = .1) +
  guides(color = guide_legend(override.aes = list(size = 5)))
```



```
ggplot(afg, aes(LON_X, LAT_Y, color = band2)) +
  geom_point(size = .1) +
  guides(color = guide_legend(override.aes = list(size = 5)))
```



It seems that there is some sort of plateau in the south western corner of the map.

Group_by in Dplyr

There are six verbs unique to the dplyr package, some that you have seen, others that you have not yet been introduced.

- **select** allows you to choose **columns**
- **filter** allows you to choose **rows**
- **mutate** allows you to change whole columns at once, and create new variables.
- **summarise** applies a function to a column that returns a single element (like `mean()`)
- **arrange** sorts the rows.
- **group_by** adds one or more group attributes to a data.frame for further grouped analysis.

Here is an short example:

```
df <- data.frame(
  city= c("Shelbyville", "Shelbyville", "Springfield",
          "Springfield", "North Haverbrook", "North Haverbrook"),
  Success= c(2,3,2,1,4,12),
  Monorail=c(0,0,0,1,0,1),
  year= c( 1994, 1994, 1994,1994, 1988, 1988)
)
```

```
df %>% summarise(mean(Success))
```

```
##   mean(Success)
## 1              4
```

```
df %>% group_by(city)%>%
  summarise(aveSuccess = mean(Success),
            changeSuccess = diff(Success))
```

```
## # A tibble: 3 x 3
##   city          aveSuccess changeSuccess
##   <fct>          <dbl>          <dbl>
## 1 North Haverbrook      8              8
## 2 Shelbyville          2.5             1
## 3 Springfield          1.5            -1
```

```
df %>% group_by(Monorail)%>%
  summarise(aveSuccess = mean(Success))
```

```
## # A tibble: 2 x 2
##   Monorail aveSuccess
##   <dbl>      <dbl>
## 1      0      2.75
## 2      1      6.5
```

```
df %>% group_by(Monorail, year)%>%
  summarise(aveSuccess = mean(Success))
```

```
## # A tibble: 4 x 3
## # Groups:   Monorail [?]
##   Monorail year aveSuccess
##   <dbl> <dbl>      <dbl>
## 1      0 1988          4
## 2      0 1994         2.33
## 3      1 1988         12
## 4      1 1994          1
```

Using Group_by in Afghanistan

Lets get some summary statistics of Afghanistan, here we use the group_by command :

```
afgbyLanguage <-  
  afg %>% group_by(LANGUAGE) %>% summarise(sumPop = sum(POPULATION))
```

The group_by command treats the variable like a factor, or an ordered categorical variable. The following commands then calculate a single number for each group. You can also use a mutate command after grouping, and this is useful if you want to retain every row of the original dataset.

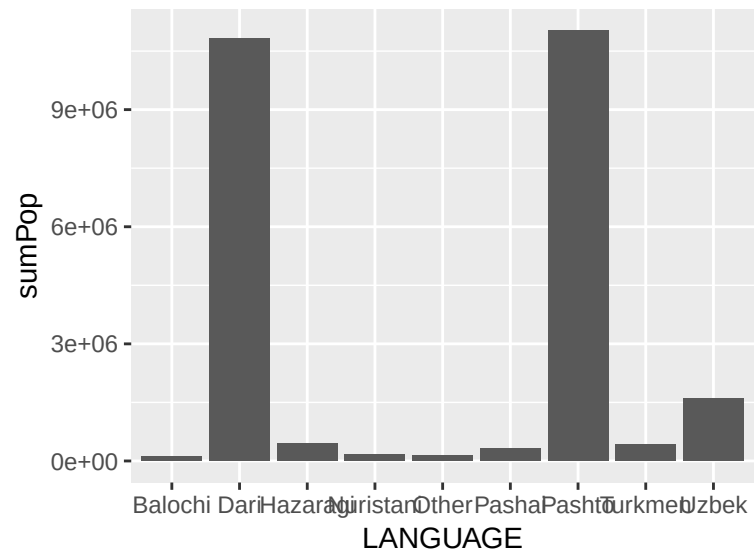
In the following, we study the size of district and provincial capital cities by language. Given that we are not interested in the other villages, I use a filter command right off.

```
afg %>%filter(!is.na(CENTER)) %>%
  group_by(LANGUAGE, CENTER) %>%
  summarise(meanPop = round(mean(POPULATION),0),
            nVillages= n())%>%
  arrange(CENTER, nVillages)
```

```
## # A tibble: 15 x 4
## # Groups:   LANGUAGE [9]
##   LANGUAGE CENTER      meanPop nVillages
##   <chr>      <chr>      <dbl>    <int>
## 1 Other      District      35451         1
## 2 Balochi    District      2037         3
## 3 Hazaragi   District      1150         3
## 4 Pashai     District      6378         5
## 5 Nuristani  District      4255         7
## 6 Turkmen    District      8354        12
## 7 Uzbek      District      6984        25
## 8 Dari       District      5756       138
## 9 Pashto     District      6470       174
## 10 Balochi   Provincial     7300         1
## 11 Hazaragi  Provincial   300600         1
## 12 Nuristani Provincial    1081         1
## 13 Uzbek     Provincial   40875         3
## 14 Dari      Provincial  263403        12
## 15 Pashto    Provincial   48810        16
```

The default bar chart is to do a frequency. If you want to do a bar chart by two variables, you need to change the stat to “identity”, or use a `geom_col`

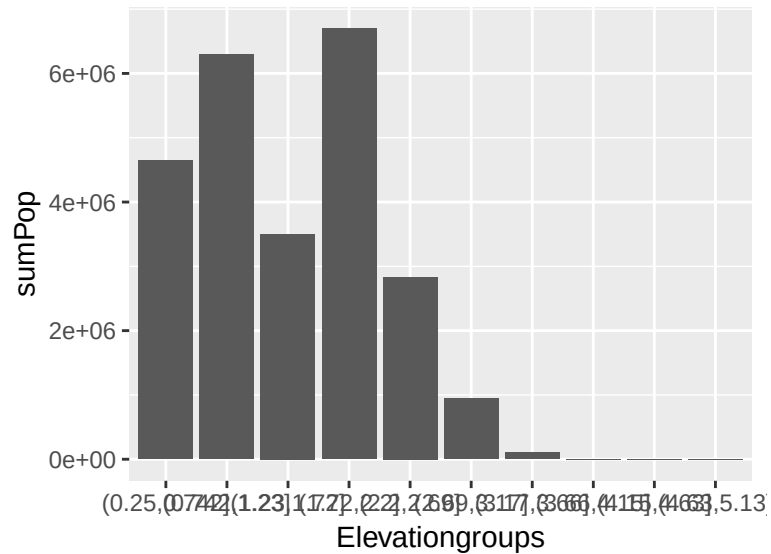
```
ggplot(afgbyLanguage, aes(LANGUAGE, sumPop)) + geom_bar(stat = "identity")
```



If you want to turn a continuous variable into a group, you can use the “cut” function.

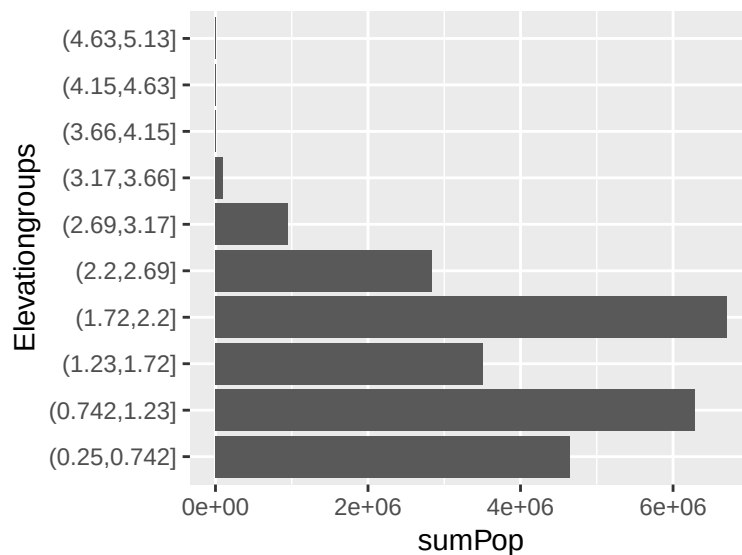
```
afgbyPopCuts <- afg %>%
  mutate(Elevationgroups = cut(ELEVATION / 1000, breaks = 10)) %>%
  group_by(Elevationgroups) %>%
  summarise(sumPop = sum(POPULATION))

ggplot(afgbyPopCuts, aes(Elevationgroups, sumPop)) + geom_bar(stat = "identity")
```



Notice that the cuts overlap one another. This occurs all the time with bar plots. A solution is to rotate with the `coord_flip` command:

```
ggplot(afgbyPopCuts, aes(Elevationgroups, sumPop)) +
  geom_bar(stat = "identity") + coord_flip()
```

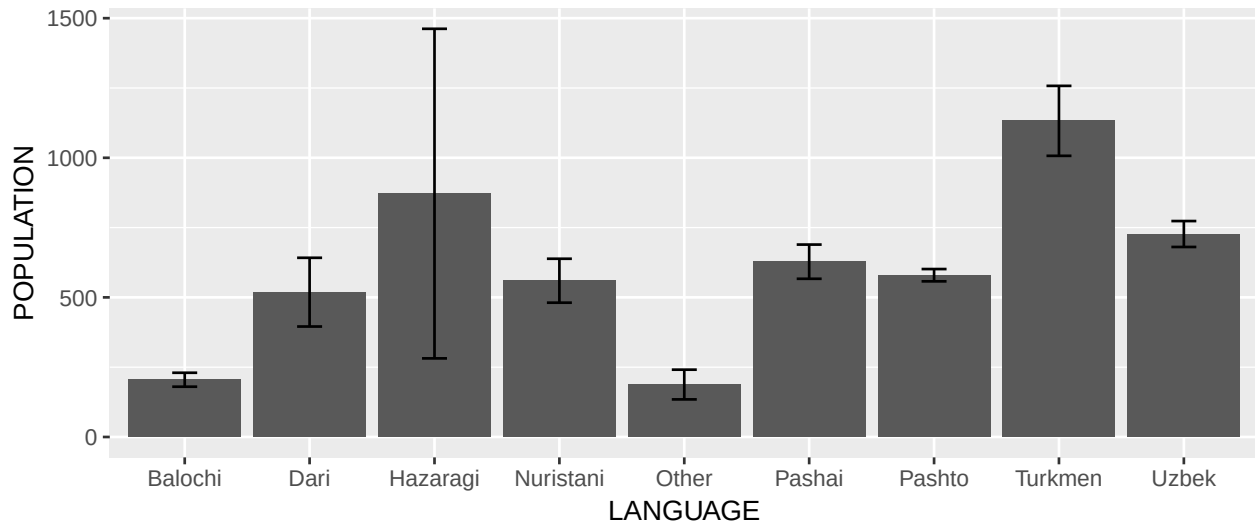


Errorbars

ggplot also has a way to automatically calculate errorbars:

Here we calculate the average population size for villages of different language groups:

```
ggplot(afg, aes(LANGUAGE, POPULATION)) +  
  stat_summary(fun.y = mean, geom = "bar") +  
  stat_summary(fun.data = mean_se,  
    geom = "errorbar",  
    width = .2)
```



Here, we are displaying the mean population of each language, and the bars indicate plus and minus one times the standard error of the sample mean. The standard error of the sample mean is calculated as follows:

$$SE(X) = \hat{\sigma} / \sqrt{N} = \sqrt{\frac{\sum (X - \bar{X})^2}{(N - 1)}} / \sqrt{N}$$

Where $\hat{\sigma}$ is the sample standard deviation (the square root of the sample variance), and N is the length of the variable, and $\bar{X}$ is the sample mean of X .

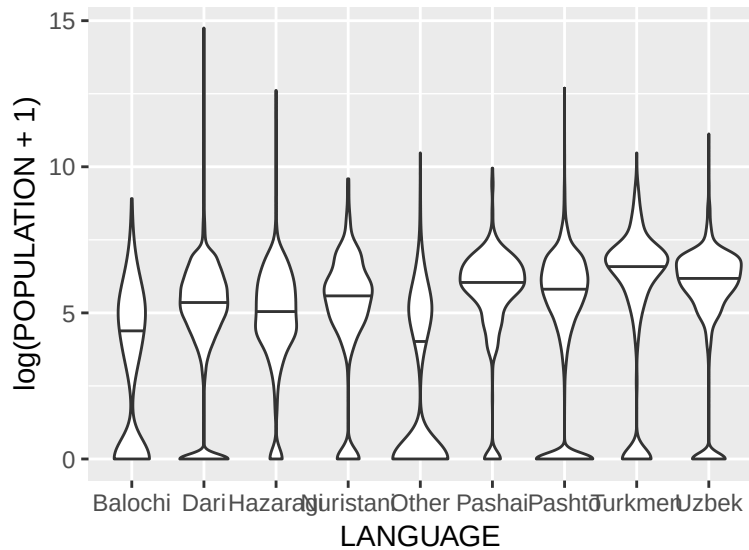
You can verify the formula by examining the contents of the `mean_se` function:

```
mean_se  
  
## function (x, mult = 1)  
## {  
##   x <- stats::na.omit(x)  
##   se <- mult * sqrt(stats::var(x)/length(x))  
##   mean <- mean(x)  
##   data.frame(y = mean, ymin = mean - se, ymax = mean + se)  
## }  
## <bytecode: 0x7fdc423e75c0>  
## <environment: namespace:ggplot2>
```

Now, one of the concerns that we might have with such a plot is that the bars draw our eyes downward. Alternatives would be to only plot the errorbars.

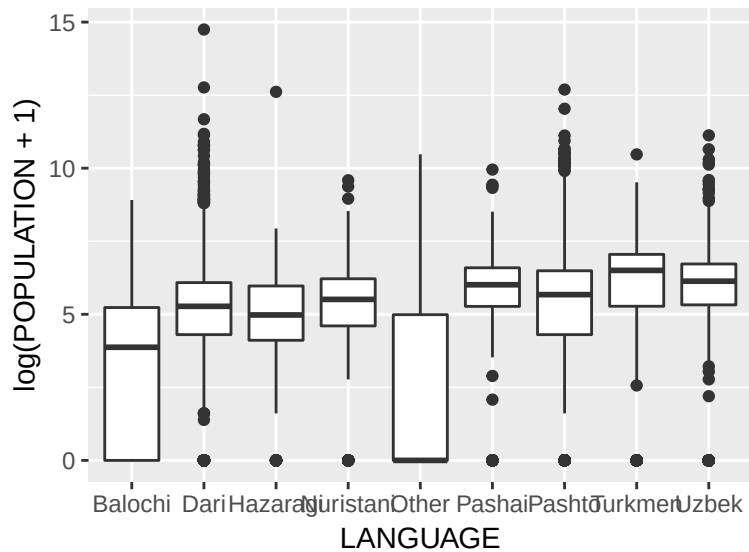
We can also show more of the distribution using a violin plot (a sideways density plot)

```
ggplot(afg, aes(LANGUAGE, log(POPULATION + 1))) +  
  geom_violin(draw_quantiles = c(.5))
```



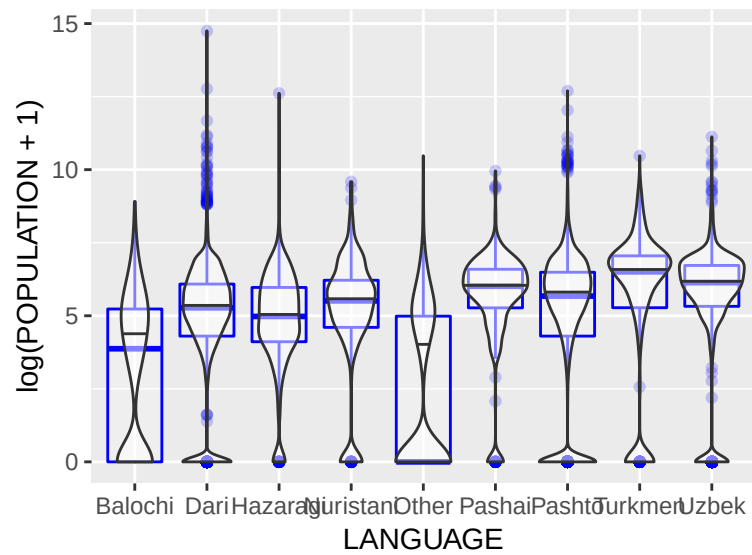
The old school version of this is the boxplot.

```
ggplot(afg, aes(LANGUAGE, log(POPULATION + 1))) +  
  geom_boxplot()
```



We can combine as well! (Don't do this, it is too hard to read, but be creative).

```
ggplot(afg, aes(LANGUAGE, log(POPULATION + 1))) +  
  geom_boxplot(alpha = .2, color = "blue") +  
  geom_violin(draw_quantiles = .5, alpha = .5)
```



Problems

- 1) Without using R, how many rows and columns does df1, df2, and df3 have?

```
dfA <- data.frame(x = 0:10, yA = rnorm(11,-1))
dfB <- data.frame(x = 5:15, yB = rnorm(11, 1))

df1 <- left_join(dfA, dfB)
df2 <- na.omit(df1)
df3 <- anti_join(dfA, dfB)
```

- 2) Turn the following code into a beautify dplyr piped code.

```
dfFondue <- data.frame(Ingredient=c("Gruyère", "vacherin", "Kirsch","Garlic",
                                   "White Wine", "cornstarch"))

dfFondue$Origin <- c("Gruyères", "Fribourg", "Zürich", "Lucern", "Jura", "Iowa")

dfFondue$Price <- c(12.99, 14.99, 15.00, 1.50, 12.00, .90)

dfFondue$Amount <- c(1, 1, .01, 1, .2, .05)

dfFondue$Proportionoftotal <- dfFondue$Price*dfFondue$Amount/sum(dfFondue$Price*dfFondue$Amount)

ggplot(dfFondue, aes(reorder(Ingredient,Proportionoftotal), Proportionoftotal))+geom_col()
```

- 3) Create a stacked barplot of the Fondue data with the ingredients sorted into alcohol (Wine and Kirsch), Cheese (Gruyere and Vacherin) and Other. Make sure the labels are correct. Fix the following ggplot commands:

- 4) Fix the following code commands:

```
votes2016<-c(-5, -4, -3, -10, 0, 3, 4, 0, 5)
votes2018<-c(0, -2, -8, -8, -9, 12, 13, 5, 3)
Primary<-c("T","C","T","T","T","C","C","C","C")
votes2020<-c(-4, -2, -5, -8, -1, 2, 8, 5, 3)

df<-data.frame(votes2016=votes2016, votes2018=votes2018, primary=Primary, votes2020=votes2020)

ggplot(df, aes(x = "votes2016", y = "votes2018")) +
  geom_point()

ggplot(df, aes(x = votes2016, y = votes2018)) +
  geom_density()

ggplot(df, aes(x = votes2016, y = votes2018)) +
  geom_point(color=primary)

ggplot(df, aes(x = votes2016, y = votes2018)) +
  geom_point(aes(color=primary))+geom_point(x=4, y=4,aes(color="blue"),shape= 3)

ggplot(df) +geom_point(aes(x = votes2016, y = votes2018))+geom_smooth()

ggplot(df) +geom_point(aes(x = votes2016, y = votes2018))+
  geom_col(filter(df, primary=="T"), aes(x = votes2016, y = votes2020) )
```